Esercizi PALG

Settimana 6

Es.1)

$G = (V, E)$

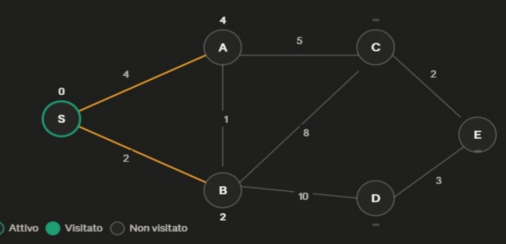
Grafo orientato con peso sia + che - ma con cicli solo positivi.

Dimostrare che Dijkstra non calcola necessariamente i cammini di costo minimo tra la radice e gli altri vertici.

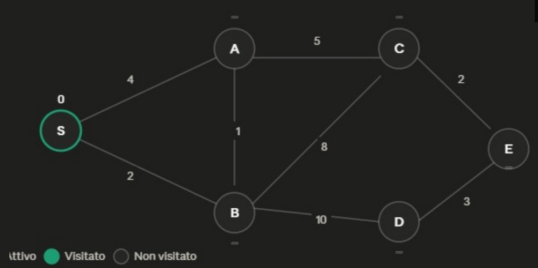
Sfruttiamo l'occasione per ripassare l'algoritmo. Questo è basato su una strategia greedy.
Che dato un grafo $G = (V, E)$ e una sorgente S: inizializza $dist[S] = 0$ e $dist[v] = \infty$.
Quindi inseriamo i nodi in una coda con priorità Q (min-heap su dist).
Finché Q non è vuota: . Si estrae u con $dist[u]$ minima for $u.adj()$ si esegue:

Il peso dell'arco tra u e v.

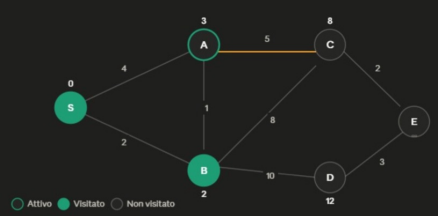
if $dist[u] + w(u,v) < dist[v]$:
$dist[v] = dist[u] + w(u,v)$



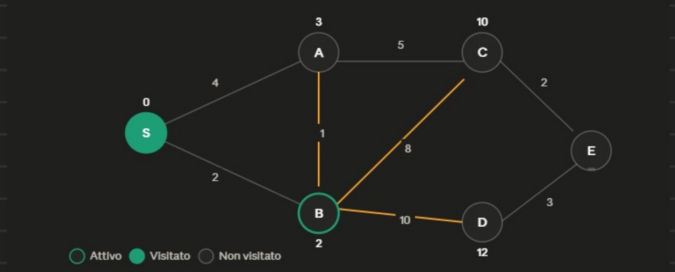
Passo 1 — Estraggo S (dist=0). Rilasso i vicini: dist[A]=min(∞, 0+4)=4, dist[B]=min(∞, 0+2)=2.



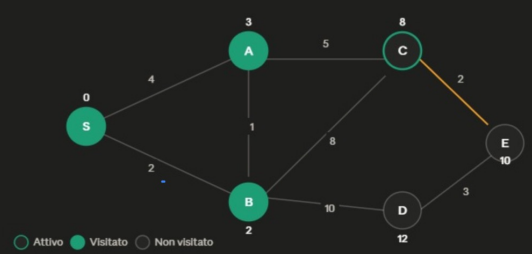
Inizializzazione: dist[S]=0, tutte le altre =



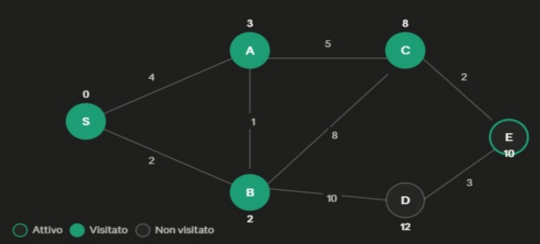
Passo 3 — Estraggo A (dist=3). Rilasso: dist[C]=min(10,3+5)=8.



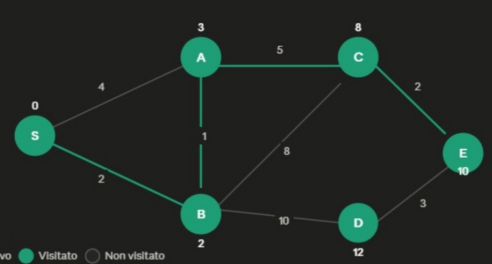
Passo 2 — S è visitato. Estraggo B (dist=2, il minore). Rilasso: dist[A]=min(4,2+1)=3, dist[C]=min(∞,2+8)=10,



Passo 4 — Estraggo C (dist=8). Rilasso: dist[E]=min(∞,8+2)=10.

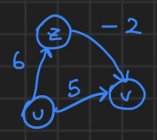


Passo 5 — Estraggo E (dist=10). Nessun vicino da rilassare ulteriormente. E è la destinazione!



Completato! Cammino minimo S→B→A→C→E = 0+2+1+5+2 = 10. Il nodo D (dist=12) non è sul cammino ottimo.

L'algoritmo funziona solo se è vera l'assunzione implicita che facciamo che $w(u,v)$ con v vicino di u è l'arco uscente da u di peso minimo allora $u \to v$ sia un cammino minimo, ma se nel grafo introduciamo archi di peso negativo potremmo avere una situazione del tipo:



Che smonta completamente l'affidabilità dell'algoritmo.

Per questi problemi si usa l'algoritmo Bellman-Ford.

ES2) Ho pensato di scrivere un algoritmo di Dijkstra che piuttosto che prendere l'arco con somma minima prenda quello con prodotto massimo.

PSEUDOCODICE:

```
                G=(V,E)
                      s∈V
network(G, s):

    for v ∈ V:
        rel[v] = 0      ] array per i valori di affidabilità
        padre[v] = NULL ] array per codificare i cammini (se troviamo un percorso migliore)
                                                          aggiorniamo il padre
    padre[s] = s
    rel[s] = 1
    Q = una coda con priorità con chiave = rel

    while Q ≠ ∅:

        u = extract(Q)
        for v in Adj(u):

            mr = rel[u] · w(u,v)
            if mr > rel[v]:
                rel[v] = mr
                padre[v] = u
                normalize(Q, v, mr)
```

mr è il prodotto tra il nodo dove ci troviamo u e i suoi vicini v ogni volta che visitiamo facciamo un check, su l'attuale rel di v e mr se mr è più affidabile aggiorniamo l'array rel e l'array padre

→ Visto che stiamo usando un heap con chiave questo si normalizza da solo

ES3) Sia $G=(V,E)$ non diretto connesso con weight.
Sia uv un arco di peso minimo. $\exists$ albero di copertura di peso minimo che contiene uv.
↳ uv ha peso minimo tra TUTTI gli archi del grafo.

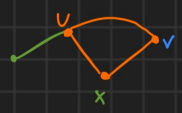
In questo caso dobbiamo considerare tutti gli MST (minimal spanning tree) possibili e verificare che ALMENO 1 contenga uv.

Costruiamo (con Kruskal o Prim) un albero di copertura minimo T

Supponiamo che $uv \notin T$.

Visto che T è un albero connesso esiste un unico cammino tra u e v, chiamiamolo P.

Se noi aggiungessimo uv a T:



si creerebbe il ciclo C.

Se noi togliessimo dal ciclo C un qualsiasi arco per esempio ux torniamo ad avere un albero di copertura $T' = (T - \{st\}) \cup \{uv\}$.
$st \to ux$

Dimostriamo che T' per un qualsiasi $st \neq uv$ con $st \in C$ è di peso minimo:

Supponiamo che $\exists\, st \mid w(st) > w(uv)$ allora T' è un albero di copertura con peso strettamente minore di T che è una contraddizione.

Sappiamo che $\nexists\, st \mid w(st) < w(uv)$ perché uv minimo.

Quindi ogni arco $\in C$ ha $w(st) = w(uv)$ quindi T' ha lo stesso peso di T ed è un albero di copertura minimo.